

Projet Lode Runner

Rapport de conception et réalisation

TALEB Yannis

COURVOISIER Thomas

Avril 2026

Table des matières

A	Introduction	3
B	Analyse fonctionnelle du sujet	4
C	Description des structures de données	5
D	Spécification des classes principales et méthodes	6
E	Architecture logicielle en couches	8
F	Algorithmes principaux	9
G	Assets du jeu et animation	14
H	Jeux de tests et Scénario de démonstration	21
I	Répartition des tâches	26

A Introduction

Présentation de Lode Runner

Avant tout, voici une courte présentation de ce qu'est Lode Runner : *Lode Runner* est un jeu vidéo de plates-formes 2D et de réflexion publié par Broderbund en 1983. Dans *Lode Runner*, le joueur incarne un membre d'un commando de l'espace chargé de récupérer l'or de son peuple dérobé par l'Empire Bungeling. Il doit pour cela explorer les cent-cinquante niveaux de la trésorerie souterraine de l'ennemi dans lesquels sont disséminés des lingots d'or.

La particularité de ce jeu réside dans le fait que le joueur ne peut pas sauter. Mais pour survivre, il peut jouer avec son environnement en creusant des trous dans les sols pour y piéger ses adversaires ou bien pour accéder à des plateformes inférieures (*Source : Wikipédia*).

Choix techniques

Bien qu'il n'ait pas été obligatoire de développer une interface graphique, nous avons fait le choix d'intégrer un rendu 2D en utilisant la bibliothèque **LibGDX**.

LibGDX est un *framework* libre et open source. Il permet de développer des jeux pour ordinateurs à partir d'une même base de code. Multiplateforme, il est compatible avec Windows, Linux et macOS.

Ce choix nous a permis de gagner du temps sur la gestion graphique et nous a permis de mieux nous concentrer sur les exigences du module Info4B : l'architecture logicielle, la communication réseau Client/Serveur et les algorithmes principaux.

B Analyse fonctionnelle du sujet

Règles de fonctionnement

Le but principal du joueur est de récolter toutes les pièces d'or du niveau tout en survivant aux ennemis. Une fois tout récupéré, il pourra quitter le niveau en utilisant l'échelle de sortie. Le jeu obéit à des règles strictes qui régissent les interactions entre les différentes entités :

- **Déplacements et Gravité** : Le joueur et les ennemis peuvent se déplacer horizontalement sur des blocs et verticalement s'ils sont situés sur des échelles. Il y a un mur tout autour qui empêche ennemis comme joueurs de quitter le jeu. De plus, le jeu intègre la gestion d'une gravité en temps réel : toute entité se trouvant au-dessus du vide tombe automatiquement jusqu'à rencontrer un sol ou une échelle.
- **Interaction avec l'environnement** : Le joueur et uniquement lui a la capacité de détruire temporairement les blocs situés d'une case en bas devant lui. Ces trous se rebouchent automatiquement après un certain délai. Cependant les sols de type "Bedrock" sont indestructibles.
- **Gestion des combats et des vies** : Le joueur commence la partie avec 5 points de vie (représentés par des cœurs en haut à droite de l'écran) et gagne un point de vie à chaque fois qu'il passe au niveau suivant. S'il rentre en contact avec un ennemi, cela lui fera perdre une vie, déclenchera une courte période d'invincibilité, et le renverra à son point d'apparition. Cependant le joueur peut se défendre en creusant des trous pour y faire tomber les ennemis. Si un trou se rebouche sur un ennemi piégé, celui-ci est éliminé et réapparaît en haut de la carte.
- **Modes de jeu (Réseau)** : L'application gère des parties en réseau incluant un mode coopératif (plusieurs joueurs collectent les trésors ensemble) et un mode combat (les adversaires peuvent être incarnés par d'autres joueurs humains).

Découpage en sous-problèmes

Nous avons découpé notre travail en trois grands sous-problèmes distincts :

- **La logique mathématique et physique (Le Moteur)** : Il fallait créer un système capable de générer le labyrinthe à partir de leur définition sous la forme de fichiers texte (Nous avons aussi la possibilité de le générer de manière aléatoire), d'appliquer la gravité à chaque instant, et de calculer les collisions de manière purement mathématique (coordonnées X et Y).
- **Le développement d'Intelligences Artificielles** : Le défi consistait à concevoir des algorithmes capables de prendre des décisions autonomes pour chasser le joueur. Nous avons fait le choix de créer plusieurs comportements (patrouilleur, IA normale et IA avec recherche du plus court chemin) s'adaptant à un environnement modifiable par le joueur.
- **La synchronisation réseau** : Le dernier sous-problème, était de réussir à partager la même partie entre plusieurs ordinateurs. Il a fallu concevoir un Serveur central qui calcule la physique, et des Clients qui reçoivent les données et les affichent à l'écran.

C Description des structures de données

La grille du labyrinthe : le tableau à deux dimensions

Pour représenter la carte du jeu, nous avons utilisé un tableau à deux dimensions `Tiles` qui se trouve dans la classe `Maze` : `private Tiles[][] map`. Cette approche nous paraissait être la plus optimale et naturelle pour un jeu basé sur une grille car :

- **Accès direct** : Il permet d'accéder facilement à n'importe quelle case via ses coordonnées X et Y . Ce qui est important dû aux nombreux calculs pour faire fonctionner les collisions et la gravité.
- **Stabilité** : La taille du niveau, c'est-à-dire sa longueur et sa largeur, ne change pas durant la partie. Donc un tableau fixe est plus économe en mémoire qu'une structure dynamique.

Gestion des entités : les listes dynamiques

Contrairement à la carte, le nombre d'acteurs présents dans le jeu peut évoluer (apparition d'ennemis). Nous utilisons donc la classe `ArrayList` de Java pour stocker les entités dans la classe `Game` :

- `List<Player> play` : gère l'ensemble des joueurs connectés.
- `List<Enemy> ene` : contient les différents types d'ennemis.
- `List<Treasure> tre` : contient les trésors à collecter.

Synchronisation réseau : `Collections.synchronizedList`

Dans la classe `Serveur`, nous devons gérer une liste de clients connectés (`ClientHandler`). Comme le serveur utilise plusieurs *threads par clients*, il y a un risque de conflit si deux clients essaient de se connecter ou de se déconnecter en même temps. Pour sécuriser ces accès, nous avons utilisé : `Collections.synchronizedList(new ArrayList<>())`. Cette structure nous garantit qu'un seul thread peut modifier la liste à la fois, évitant ainsi les erreurs de type `ConcurrentModificationException`.

Algorithmique de l'IA : la File (Queue)

Parmi nos 3 types d'ennemis, il y en a un plus "avancé" (`EnemyIA`), nous avons implémenté un algorithme de parcours en largeur (BFS). Cet algorithme nécessite une structure de **File** (*First-In, First-Out*) pour explorer les cases adjacentes une par une. Nous avons utilisé l'interface `Queue` implémentée par une `LinkedList` : `Queue<NoeudChemin> file = new LinkedList<>()`.

- L'IA met à la fin de la File les cases adjacentes à sa position.
- Étant donné qu'elle traite toujours l'élément situé au début de la File, l'IA est contrainte d'explorer **toutes** les cases situées à 1 pas de distance, avant de commencer à explorer les cases à 2 pas de distance, et ainsi de suite.

Cette technique nous garantit que lorsque l'algorithme croisera pour la première fois la case du joueur, il n'existera pas d'autre chemin plus court (puisque tous les chemins plus courts ont déjà été explorés et éliminés).

D Spécification des classes principales et méthodes

Dans cette section, nous détaillerons l'organisation interne de notre code.

Hiérarchie des entités (Héritage et Polymorphisme)

Toutes les entités du jeu (joueurs, ennemis, trésors) découlent d'une structure commune qui nous permet de manipuler les coordonnées :

- **Entity** : Classe mère contenant les coordonnées en private x et y ainsi que leurs accesseurs (*getters/setters*).
- **Character** : Classe abstraite héritant d'**Entity**. Elle introduit la notion de **Direction** (Haut, Bas, Gauche, Droite) et les méthodes de déplacement de base (**right()**, **left()**, etc.).
- **Player** : Représente l'utilisateur. Elle ajoute la gestion des (5) points de vie (**hp**) et de l'invincibilité temporaire quand on respawn.
- **Enemy** : Classe abstraite pour les adversaires. C'est ici qu'on trouve la méthode **free** (pour savoir si l'ennemi est piégé dans un trou) et la méthode abstraite **mouvement()** que chaque type d'ennemi doit implémenter différemment.

La classe Game

La classe **Game** centralise toutes les données et les règles :

- **Méthode sec()** : C'est la méthode qui "met à jour" le jeu. Elle est appelée à chaque "tick" du serveur et déclenche successivement la gravité, le mouvement des ennemis, la détection des collisions et l'ouverture de la sortie.
- **Méthode fall()** : Gère la physique de chute pour n'importe quel **Character**, qu'il s'agisse d'un joueur ou d'un ennemi.
- **Gestion des fichiers** : Elle contient les méthodes **saveToFile()** et **loadFromFile()** permettant de traduire l'état du jeu en un format texte (**level.txt**) pour la sauvegarde des données.

Infrastructure Réseau

La communication est assurée par trois classes :

- **Serveur** : Elle s'occupe de la boucle du jeu principale (**gameLoop()**) et accepte les nouvelles connexions dans des *threads* séparés.
- **ClientHandler** : S'exécute sur le serveur pour chaque joueur connecté. Il regarde les actions envoyées par le client (comme le déplacement) et lui renvoie l'état complet du jeu (**sendGame()**), c'est la classe entre client et serveur.
- **Client** : C'est le côté utilisateur, il regarde les mises à jour en continu dans un *thread* dédié (**listen()**) et transmet les entrées clavier au serveur via la méthode **action()**.

Diagramme de classe

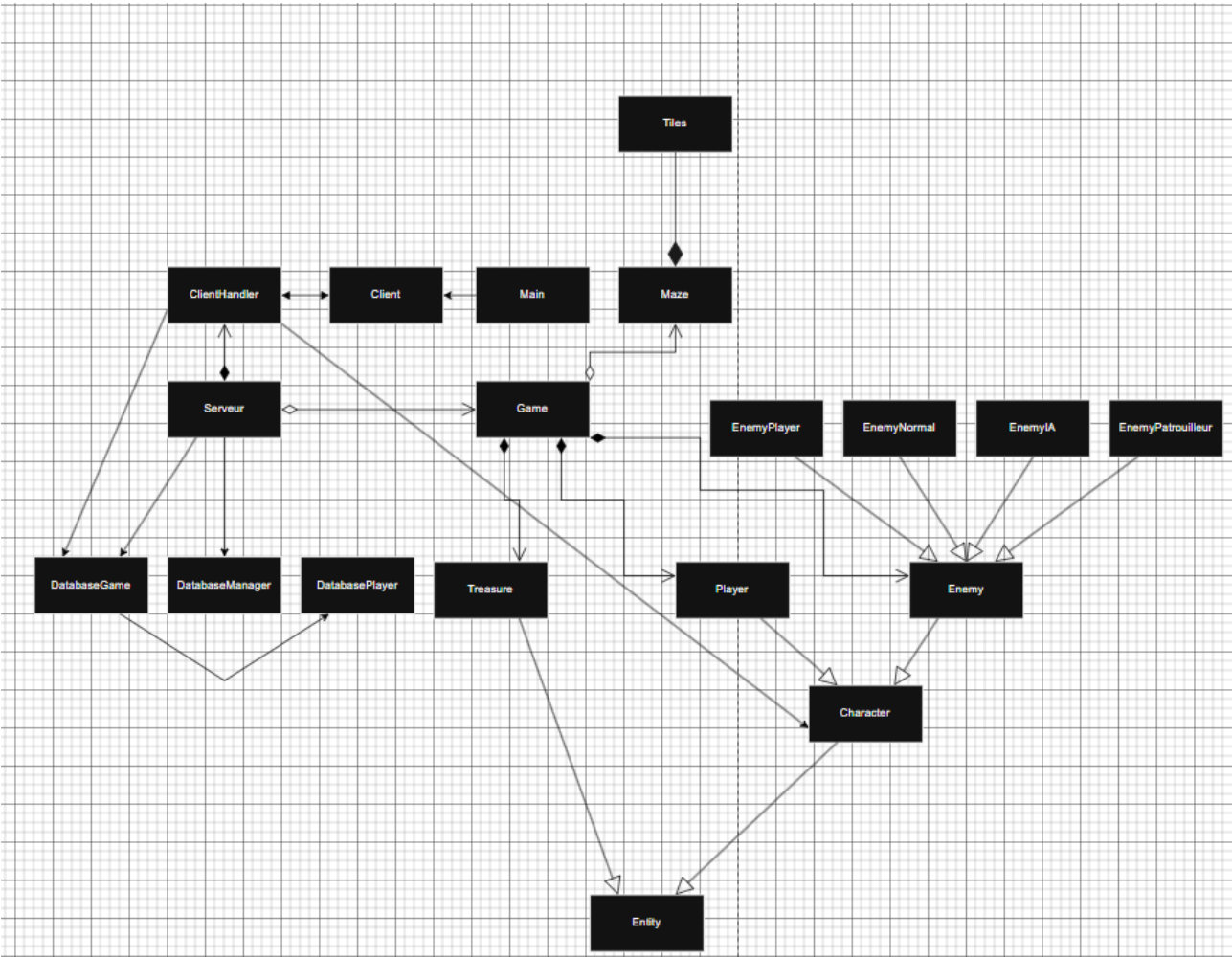


FIGURE 1 – Diagramme de classe

E Architecture logicielle en couches

Conformément aux exigences du module Info4B, nous avons conçu notre application en respectant une séparation stricte des responsabilités. Cette conception s'inspire de l'architecture en couches d'un système d'exploitation, où chaque niveau fournit des services au niveau supérieur.

Matériel

Dans un système d'exploitation, cette couche interagit avec le matériel physique (hardware). Dans notre projet, c'est la bibliothèque **LibGDX**. Elle agit comme une interface entre notre code Java et les composants de l'ordinateur (carte graphique, clavier). Par exemple, dans la classe **Main**, la méthode `inputPlayer()` utilise `Gdx.input` pour capter la touche du clavier que l'utilisateur presse.

Couche Noyau

Le noyau est le cœur du système, il gère les ressources de l'ordinateur et permet aux différents composants - matériels et logiciels — de communiquer entre eux. Dans notre projet, c'est le package **engine** et particulièrement la classe **Game** qui joue ce rôle. Elle détient l'état du monde (la grille du labyrinthe, les listes d'entités). Sa méthode `sec()` agit comme un planificateur : elle applique les règles du jeu (gravité, détection des collisions, gestion des points de vie).

Communication (IPC)

Dans un OS, les processus doivent pouvoir communiquer via des processus de communication d'IPC (*Inter-Process Communication*). Dans notre projet cette fonction est assurée via le package **network** :

- **Le Serveur** (`Serveur.java` et `ClientHandler.java`) : Il reçoit les requêtes d'action des clients, demande au noyau (**Game**) de les valider et d'exécuter les mouvements, puis met à jour le nouvel état global du jeu.
- **Le Client** (`Client.java`) : Il "écoute" le réseau pour réceptionner les données et mettre à jour la version locale du jeu.

Couche Application

C'est la couche visible par l'utilisateur, que l'on pourrait comparer à un logiciel d'un ordinateur. Dans notre jeu, c'est la classe **Main** qui représente cet espace utilisateur. Son rôle principal est d'utiliser les données qui ont été validées par le noyau pour les dessiner à l'écran à 60 images par seconde, grâce à sa méthode `render()` et à l'outil **SpriteBatch**.

F Algorithmes principaux

Cette partie détaille la logique des fonctionnalités principales de notre moteur de jeu.

1. L’algorithme de Gravité et de Chute

Dans *Lode Runner*, la gravité est une force constante qui s’applique à la fois aux joueurs et aux ennemis. L’algorithme de chute est géré par les méthodes `gravity()` et `fall()` de la classe `Game`.

À chaque cycle d’horloge du jeu (*tick*), l’algorithme effectue les vérifications suivantes pour chaque entité :

1. **Vérification des limites** : Si l’entité se trouve déjà tout en bas de la carte ($Y + 1 \geq \text{hauteur max}$), la chute est annulée.
2. **Collision avec une autre entité** : L’algorithme parcourt la liste des ennemis. Si l’entité qui tombe atterrit exactement sur les coordonnées d’un ennemi situé une case plus bas, la chute s’arrête.
3. **Vérification du sol** : L’algorithme analyse la case située sous l’entité ($X, Y + 1$).
 - Si la case est du vide (`Type 0`).
 - Ou si la case est un mur en briques détruit par le joueur (`Type 1` avec `State = false`).
4. **Action** : Si les vérifications du sol sont remplies, les coordonnées de l’entité sont mises à jour ($Y = Y + 1$). Dans le cas d’un ennemi tombant dans un trou détruit, un état `free = false` lui est appliqué pour signifier qu’il est piégé, et le joueur gagne 10 points.

Cet algorithme est peu coûteux en calculs, ce qui permet au serveur de l’exécuter des dizaines de fois par seconde sans ralentir la partie.

```
1 public void fall(Character c){
2     if(c.getY()+1 >= maze.getHeight()){
3         return;
4     }
5
6     for (Enemy e: ene){
7         if(c.getX() == e.getX() && c.getY() == e.getY()-1 && e.getFree()==
false){
8             return;
9         }
10    }
11
12    if (c instanceof Enemy){
13        Enemy e = (Enemy) c;
14        if(maze.getTile(c.getX(), c.getY()).getType()==1 && maze.getTile(c.
getX(), c.getY()).getState()==false && e.getFree()){
15            this.score += 10;
16            e.setFree(false);
17            e.setTimeToRespawn(60);
18            return;
19        }
20    }
21    Tiles tileUnder = maze.getTile(c.getX(), c.getY()+1);
22    if((tileUnder.getType()==0 || (tileUnder.getType()==1 && tileUnder.
getState()==false))){
23        c.down();
24    }
25 }
```

Listing 1 – Application de la gravité dans `Game.java`

2. Algorithme de Génération Procédurale du Labyrinthe

Étant donné que nous avons fait le choix de niveaux infinis pour notre jeu (il n'y a pas un nombre fixe de niveaux à faire), il fallait que chaque niveau soit différent des autres. Pour ce faire, nous avons implémenté un algorithme de génération procédurale dans la méthode statique `generation()` de la classe `Maze`. Ce n'est pas une création purement aléatoire car cela pourrait bloquer le joueur, notre algorithme suit une logique de construction par "étapes de montée". Voici comment cela fonctionne :

L'algorithme se déroule selon les phases suivantes :

1. **Initialisation et Bordures** : Une taille aléatoire est définie (entre 20 et 29 cases). La méthode `bord()` remplit les contours de la matrice avec des briques de type "Bedrock" (incassables) qui est la limite physique du jeu.
2. **Construction des plateformes** : L'algorithme commence au bas de la carte ($y = \text{hauteur} - 1$) et "monte" vers le sommet par itérations successives. À chaque étape, il définit une hauteur de saut aléatoire (h) pour placer le prochain niveau de plateformes.
3. **Placement des Plateformes** (`plat()`) : Pour chaque étage, une plateforme de briques cassables est générée avec une longueur aléatoire. L'algorithme s'assure que ces plateformes ne traversent pas les bordures de la carte.
4. **Liaison par Échelles** (`ladder()`) : Une fois une plateforme posée, l'algorithme choisit un point aléatoire sur celle-ci pour y construire une échelle qui descend jusqu'à rencontrer le sol précédent. Comme cela, on sait qu'il existe toujours un chemin vertical entre deux étages.
5. **Point de sortie** : Lorsque l'algorithme atteint le haut de la carte, il place aléatoirement la sortie du niveau et construit l'échelle finale qui permettra au joueur de s'échapper une fois tous les trésors collectés.

Cela permet de générer des niveaux différents sans risque de se bloquer.

```
1 public static Maze generation(){
2     int size = (int)(Math.random()*10)+20;
3     Maze mazegenerator = new Maze(size, size, 0);
4     mazegenerator.bord();
5     int y = mazegenerator.getHeight()-1 ;
6     while (mazegenerator.getExit()==0){
7         int x = (int) (Math.random()*(mazegenerator.getWidth()-2))+ 1;
8         int h = (int) (Math.random()*2)+2;
9         y=y-h;
10        if (y <= 1){
11            mazegenerator.setExit(x);
12        }else {
13            int l = (int)(Math.random()*(mazegenerator.getWidth()-4)) +
14            3 ;
15            int[] borneplat=mazegenerator.plat(x, y, l);
16            int xLadder = borneplat[0] + (int) (Math.random()*(borneplat
17            [1]-borneplat[0]));
18            mazegenerator.ladder(xLadder, y);
19        }
20    }
21    mazegenerator.ladder(mazegenerator.getExit(), 0);
22    return mazegenerator;
23 }
```

Listing 2 – Logique de l'algorithme de génération dans `Maze.java`

3. Algorithmes d'Intelligence Artificielle (Les Ennemis)

Dans notre version de Lode Runner, il existe plusieurs types d'ennemis. En utilisant le polymorphisme, le jeu se contente d'appeler la méthode `mouvement(this)` sur chaque entité ennemie, sans se soucier de son type. L'algorithme interne de l'ennemi calcule alors la meilleure action et renvoie une `Direction`.

Voici les 3 types d'ennemis que le joueur croisera durant sa run :

1. **Le Patrouilleur (EnemyPatrouilleur)** : Cet ennemi a 2 états avec des comportements différents : `PATROUILLE` et `POURSUIT`.
 - En mode `PATROUILLE`, il avance dans une direction jusqu'à rencontrer un mur (la `bedrock`) ou du vide (un vide naturel, c'est-à-dire pas un bloc cassé par le joueur), puis fait demi-tour.
 - À chaque itération, il regarde s'il est sur la même ligne horizontale que le joueur : s'il détecte un joueur et qu'aucun mur ne les sépare, il passe en mode `POURSUIT` et se dirige vers le player.
2. **L'Ennemi Basique (EnemyNormal)** : Cet ennemi utilise un algorithme simple de recherche d'échelles. Il a deux priorités :
 - *Alignement vertical* : Il compare sa coordonnée Y avec celle du joueur. S'il doit monter, il cherche une échelle en balayant la plateforme sur laquelle il se trouve. S'il doit descendre, il cherche une échelle ou un trou.
 - *Alignement horizontal* : Une fois sur la même ligne horizontale (Y) que le joueur, il se dirige simplement vers lui sur l'axe X.
3. **L'Ennemi Avancé (EnemyIA)** : Cette IA est la plus dure. Contrairement à l'ennemi basique, elle utilise un algorithme de théorie des graphes : le parcours en largeur (**BFS**). À chaque déplacement, l'algorithme simule une exploration de la grille dans toutes les directions dans lesquelles on peut y aller (en tenant compte de la gravité et des échelles) jusqu'à trouver les coordonnées du joueur. Il prend ensuite la toute première étape de ce chemin le plus court et renvoie la direction de celle-ci. De plus, si le chemin le plus court inclut de devoir se laisser tomber, alors ce chemin va être utilisé, ce qui fait que l'ennemi peut intentionnellement se laisser tomber pour arriver le plus rapidement possible au joueur.

Note : L'EnemyPlayer, il ne possède aucun algorithme décisionnel car il est destiné au mode combat, où la méthode `mouvement()` utilise les commandes envoyées par un joueur humain via le réseau.

De plus, les 3 types d'ennemis sont plus lents que le joueur. Pour des raisons de difficulté. Si les ennemis avaient eu la même vitesse que le joueur, le jeu aurait été beaucoup trop compliqué.

```

1 public void movementEnemy(){
2     for(Enemy e : this.ene){
3         if (e.getFree() && e.getState()) {
4             Direction intention = e.mouvement(this);
5
6             if (intention == Direction.RIGHT) {
7                 moveCharacterRight(e);
8             }
9             else if (intention == Direction.LEFT) {
10                 moveCharacterLeft(e);
11             }
12             else if (intention == Direction.UP) {
13                 moveCharacterUp(e);
14             }
15             else if (intention == Direction.DOWN) {
16                 moveCharacterDown(e);
17             }
18         }
19     }
20 }

```

Listing 3 – Utilisation du polymorphisme pour le mouvement des IA

4. Algorithme de Boucle de Jeu et Synchronisation

La boucle de jeu du serveur, implémentée dans la méthode `gameLoop()` de la classe `Serveur`. Elle sert à éviter que le jeu ne s'exécute trop rapidement. De plus, le serveur cadence le temps :

1. **Cadencement (Tick-rate)** : La boucle tourne en continu mais impose une pause (`Thread.sleep(50)`) pour chaque itération, ce qui fixe la fréquence de mise à jour du serveur à 20 ticks par seconde.
2. **Ordonnancement des actions** : Un compteur `tick` est incrémenté. Au lieu de tout calculer en même temps, l'algorithme se répartit la charge :
 - Tous les 2 ticks (`tick % 2 == 1`), le moteur physique est appelé via `g.sec()` pour appliquer la gravité et les collisions.
 - Tous les 35 ticks, un signal d'activation des ennemis est envoyé via `g.activEnemy()`. C'est un choix que nous avons décidé de faire. Cela permet de faire apparaître les ennemis les uns après les autres.
3. **Diffusion de l'état** : À la fin de chaque cycle, le serveur verrouille la liste des clients (`synchronized`) et diffuse l'objet `Game` mis à jour à tous les joueurs connectés.

```

1 public void sec(){
2     gravity();
3     movementEnemy();
4     playerEnemyCol();
5     playerTreasureCol();
6     decrementPlayerInvTimer();
7     decrementEnemyTimer();
8     decrementTileTimer();
9     canEscape();
10 }
11
12 public void activEnemy(){
13     boolean activ = false;
14     int i = 0;
15     while(i < this.getEne().size() && !activ){
16         if(!this.getEne().get(i).getState()){
17             this.getEne().get(i).setState(true);
18             activ = true;
19         }
20         i++;
21     }
22 }

```

Listing 4 – chronometre du jeu

G Assets du jeu et animation

L'aspect visuel de notre projet repose sur l'utilisation de *spritesheets* et sur un système de rendu dynamique géré par la bibliothèque LibGDX.

1. Les planches de Sprites (Assets)

Tous les éléments visuels du jeu (le joueur, les ennemis, les briques, les échelles et les trésors) ont été conçus ou regroupés sur des planches 2D. Voici un exemple avec le personnage courant à droite :



FIGURE 2 – spritesheet de la course du personnage

L'animation en 2D n'est qu'une succession rapide d'images (les *frames*). Pour donner vie à nos entités (le Héros, les Ennemis et les cœurs), nous avons découpé la planche de sprites en plusieurs séquences. Par exemple pour le player qui court sur la droite il y a 6 frames :

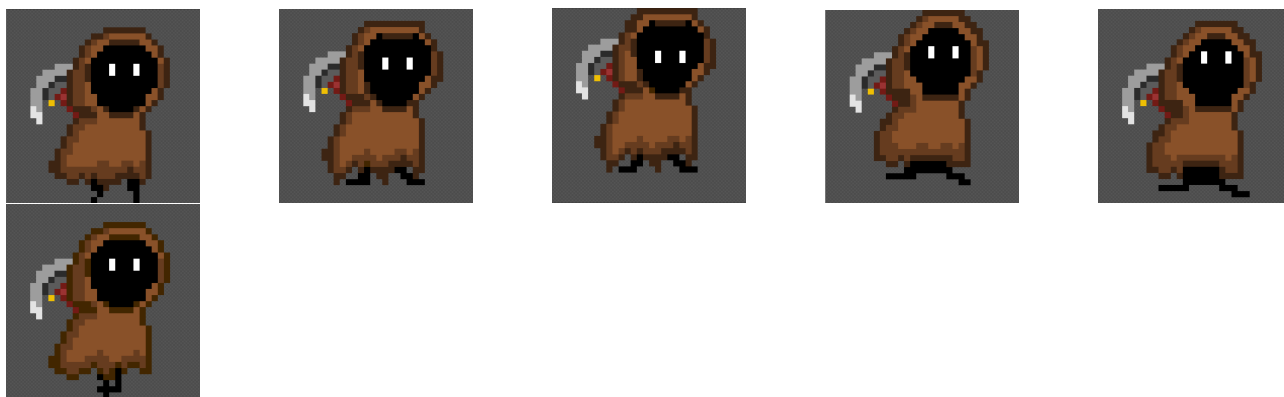


FIGURE 3 – Décomposition des frames pour l'animation de déplacement vers la droite.

Pour faire l'animation du personnage qui court vers la gauche, il a juste fallu faire une symétrie axiale :



FIGURE 4 – symétrie axiale pour la course vers la gauche

Maintenant que nous avons expliqué le principe, voici la liste de tous les éléments se trouvant dans le jeu :

Les points de vie :



FIGURE 5 – Animation des points de vie

L'animation possède 6 frames et cela donne l'impression que les cœurs sont en rotation.

Les différents types de blocs



FIGURE 6 – Les différents types de blocs.

Nous avons dans l'ordre : L'échelle, la bedrock (la limite du jeu) qui est le bloc incassable, le sol et pour finir le sol creusé par le player.

Les pièces d'or



FIGURE 7 – Animation des pièces d'or

Les pièces ont à la différence du cœur 7 frames et elles donnent l'impression qu'elles sont en rotation.

L'invincibilité du joueur

Pour rappel le joueur est invincible quand il réapparaît après avoir subi un dégât de la part d'un ennemi. Pour différencier le Player invincible, nous avons fait le choix de lui mettre une animation pour que l'utilisateur sache qu'il ne peut plus subir de dégâts.



FIGURE 8 – Animation d'invincibilité

Et quand il tombe, il possède aussi une animation spéciale :



FIGURE 9 – Animation d'invincibilité

Monter les échelles et tomber

Il n'y a qu'une seule frame qui s'affiche quand le Joueur monte les échelles et tombe.

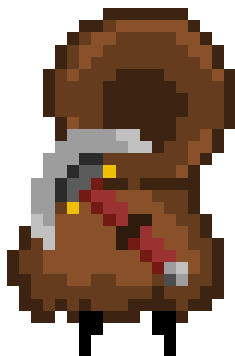


FIGURE 10 – Le joueur quand il monte/descend l'échelle

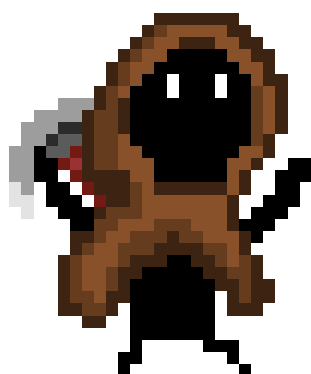


FIGURE 11 – Le joueur quand il tombe

Les ennemis

Les ennemis ont seulement deux animations.

— Quand ils se déplacent sur la droite ou sur la gauche.



FIGURE 12 – Le joueur quand il tombe

Le choix a été de faire des ennemis n’ayant pas de jambes, ce qui est assez dur à animer en 64x64 pixels. C’est pour cela que j’ai fait le choix de faire des ennemis de type fantômes.

2. Mécanique de découpage des Spritesheets

Pour animer nos entités, nous utilisons la classe `Animation<TextureRegion>` fournie par LibGDX. Au lieu de charger manuellement des dizaines d’images individuelles, nous avons conçu une méthode nommée `creerAnimation()` qui automatise le traitement visuel.

Le processus de traitement se déroule en trois étapes :

1. **Chargement et découpage** : La méthode charge la *spritesheet* complète sous forme de `Texture`, puis utilise l’outil `TextureRegion.split()` pour la diviser mathématiquement en une matrice 2D, en se basant sur la taille d’une *frame* : qui est de 32x32 pixels pour le joueur et la pièce et 64x64 pixels pour tout le reste.
2. **Aplatissement** : La méthode parcourt ensuite cette matrice à l’aide d’une double boucle pour en extraire un tableau, en s’arrêtant exactement au nombre de *frames* nécessaires pour l’animation.
3. **Instanciation** : Un objet `Animation` est généré en liant ce tableau d’images à un intervalle de temps d’affichage fixe qui est de 0.1 seconde par image soit 10 fps. 10 fps est suffisant, étant donné que la spritesheet possède que 6-7 frames différentes.

3. Synchronisation par DeltaTime

Pour garantir une vitesse d’animation constante, indépendamment de la puissance de l’ordinateur (les *FPS*). Nous n’utilisons pas le compteur de *ticks* du Serveur, mais un accumulateur

de temps nommé `stateTime`. À chaque cycle de rendu graphique (`render()`), le temps exact écoulé depuis l’affichage de l’image précédente est récupéré via `Gdx.graphics.getDeltaTime()` et ajouté à cet accumulateur.

Au moment de dessiner le personnage (`drawPlayer()`), le système regarde cet accumulateur via la fonction `getKeyFrame(stateTime, true)`. Le moteur calcule alors automatiquement quelle image du tableau afficher à cet instant précis, garantissant une animation fluide.

H Jeux de tests et Scénario de démonstration

Scénario 1

Le joueur casse une brique pour y bloquer un ennemi dans le sol, l'ennemi meurt et réapparaît en haut :

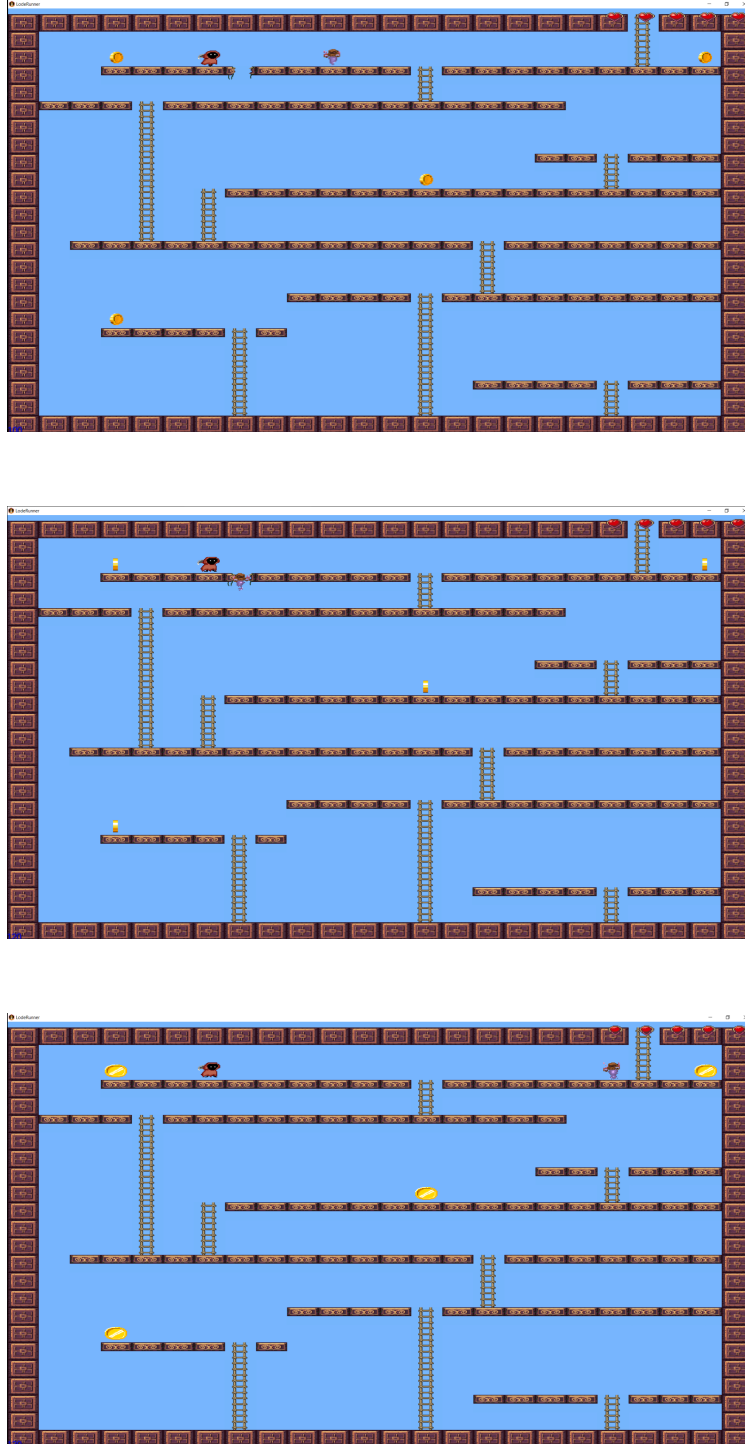


FIGURE 13 – Scenario 1

Scénario 2

Le joueur récupère toutes les pièces du niveau et passe au niveau suivant :

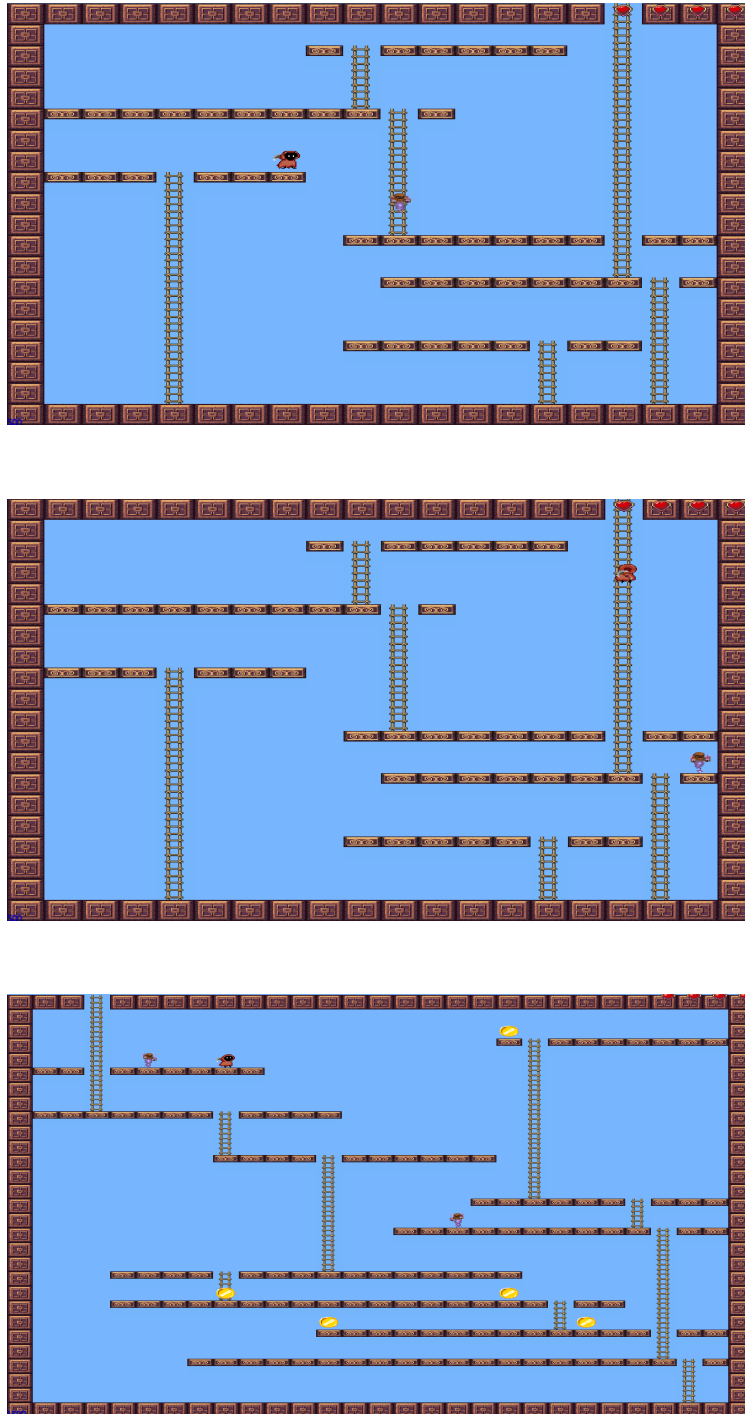


FIGURE 14 – Scenario 2

Scénario 3

2 joueurs sont connectés sur la même partie (mode coopération), le joueur contrôlé est en marron tandis que l'allié est en rouge, pour se différencier.

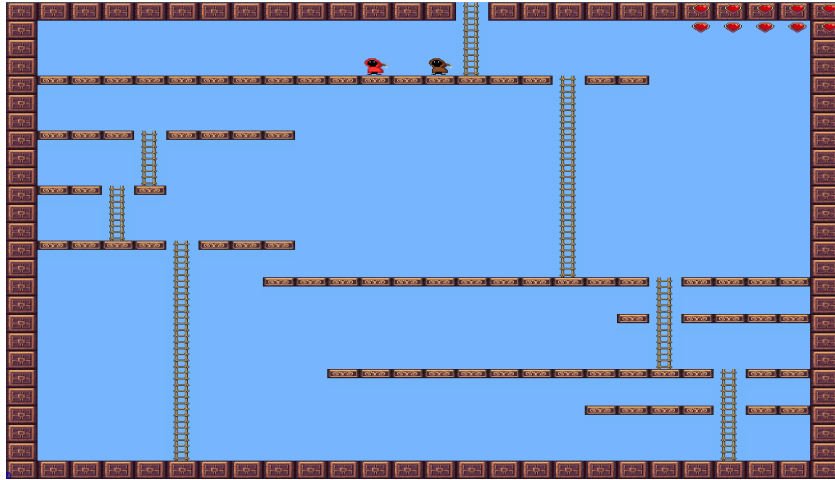


FIGURE 15 – deux joueurs en coop

Scénario 4

Le joueur contrôle un ennemi

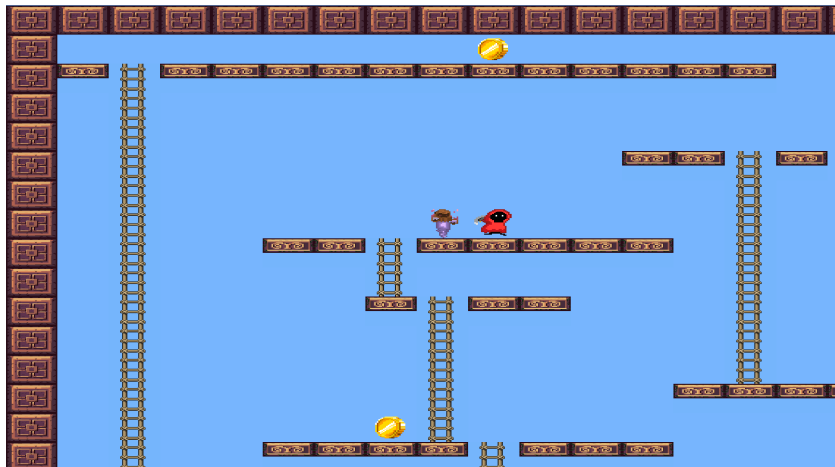


FIGURE 16 – L'utilisateur contrôle un ennemi

Scénario 5

La map est chargée par un fichier texte sauvegardé en amont :

```
21 21 13 410 1 17 COOP
=====H=====
=       H       =
=       H       =
=   #####   =
=       H       =
=       H       =
-#####H#####
=       H H      =
=       H H      =
=   ##### H      =
=       H H      =
=   HHH##### =
=       H H      =
=       H H      =
=   #####H#####
=       H       =
=       H       =
=       HHH      =
=       H       =
=       H       =
=====
Play
19 10 4 Player 0
Ene
13 2 true 0 true 113 2 true 0 true 1
Tre
16 2 false
10 2 true
10 8 true
6 5 true
16 10 true
```

FIGURE 17 – Image du niveau en fichier txt

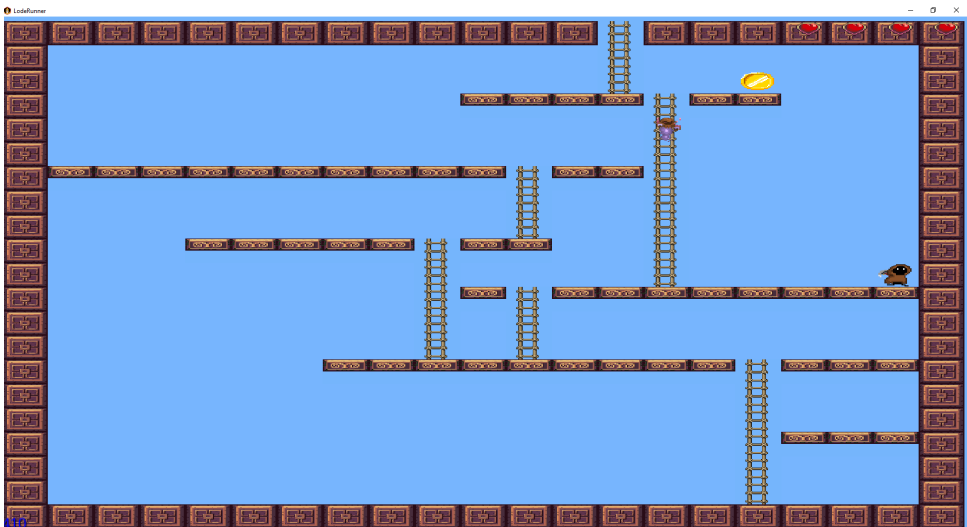


FIGURE 18 – Image du niveau

Le score

De plus, quand on perd (game over) les scores des 10 meilleurs partie du mode jeux actuelle est affiché à l'écran.



FIGURE 19 – Image du niveau

Conclusion

En conclusion, le développement de Lode Runner était très enrichissant. Nous avons appris à faire plusieurs choses comme la conception d'un moteur physique en passant par l'implémentation d'une l'intelligence artificielle et sans oublier la synchronisation réseau, ce projet nous a permis de mettre en pratique beaucoup de notions abordées dans le module Info4B. Certes, le jeu aurait pu être amélioré. Il y a beaucoup d'axes d'amélioration, comme un menu, mais nous somme assez fière de ce que nous avons réalisé.

I Répartition des tâches

Membre du Groupe	Tâches réalisées
<i>COURVOISIER Thomas</i>	Réseau & Serveur
<i>COURVOISIER Thomas</i>	génération procédurale de la map
<i>COURVOISIER Thomas</i>	Déroulement de la partie
<i>COURVOISIER Thomas</i>	Base de données
<i>Travail conjoint</i>	Moteur du jeu (Algorithmique de la gravité, gestion des collisions,...)
<i>TALEB Yannis</i>	Conceptions des 3 types ennemies
<i>TALEB Yannis</i>	Assets du jeu et animation
<i>TALEB Yannis</i>	Rapport du projet

FIGURE 20 – Répartition des tâches